

CONSTRUTOR E DESTRUTOR

Prof. André Backes

Definição

- O construtor e o destrutor são métodos especiais que permitem a criação e a destruição de objetos de uma classe de maneira controlada
 - Eles são parte da classe, mesmo que não sejam definidos.
- São úteis para
 - Inicializar os atributos de um objeto quando ele é criado
 - Executar tarefas de limpeza ou liberação de recursos quando o objeto é removido da memória

Construtor

- Método chamado quando uma nova instância do objeto é criada
 - O nome do construtor é o mesmo da classe, sem tipo de retorno
 - É automaticamente chamado no momento em que criamos um novo objeto do tipo Pessoa

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    // Construtor  
    Pessoa(string nome, int idade) {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p("Maria", 25);  
  
    return 0;  
}
```

Construtor

- O construtor funciona como um inicializador de atributos
 - Garante que o atributo sempre existe
- É responsável pela
 - Alocação de recursos do objeto
 - Definição inicial dos estados dos atributos

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    // Construtor  
    Pessoa(string nome, int idade) {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p("Maria", 25);  
  
    return 0;  
}
```

Construtor

- Podemos usar o construtor para
 - Executar funções de configuração
 - Verificação de erros nos valores de inicialização
 - Abertura de um arquivo
 - Alocação de memória para alguma estrutura de dados
 - etc.

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    // Construtor  
    Pessoa(string nome, int idade) {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p("Maria", 25);  
  
    return 0;  
}
```

Construtor | Funcionamento

- O compilador
 - Busca um construtor que aceite uma **string** e um **int** como parâmetros
 - Vários construtores podem existir
 - Caso não exista, fará a conversão implícita de argumentos
 - Chama o construtor mais parecido

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    // Construtor  
    Pessoa(string nome, int idade) {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p("Maria", 25);  
  
    return 0;  
}
```

Construtor | Funcionamento

- Em tempo de execução
 - Alocação da memória do objeto
 - Construtor selecionado é chamado
 - Atributos são inicializados na ordem em que foram declarados na classe,
 - Independentemente da ordem listada no inicializador
 - Executa o corpo da função construtora

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    // Construtor  
    Pessoa(string nome, int idade) {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p("Maria", 25);  
  
    return 0;  
}
```

Construtor | Funcionamento

- No corpo da função construtora podemos incluir qualquer comando que julgemos necessário para a criação correta do objeto
- Podemos
 - Executar funções de configuração
 - Verificação de erros nos valores de inicialização
 - Abertura de um arquivo
 - Alocação de memória para alguma estrutura de dados
 - etc

Construtor | Sobrecarga

- O construtor suporta sobrecarga
- Eles podem ser
 - Padrão: sem parâmetros, inicializa com valores pré-definidos
 - Parametrizado: inicializa atributos com valores específicos
 - De cópia: cria um novo objeto como uma cópia de outro já existente

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
  
public:  
    // Construtor padrão  
    Pessoa() {  
        nome = "Indefinido";  
        idade = 0;  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
    // Construtor parametrizado  
    Pessoa(string n, int i) {  
        nome = n;  
        idade = i;  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p1;  
    Pessoa p2("Maria", 25);  
  
    return 0;  
}
```

Construtor | Inicialização

- Para a inicialização de atributos, podemos
 - Usar a atribuição
 - Definir uma lista de inicialização
 - Opção mais interessante
 - Usar valores-padrão

```
// Construtor com atribuição
Pessoa(string n, int i){
    nome = n;
    idade = i;
    cout << "Nome: " << nome << ", Idade: " << idade << endl;
}

// Construtor com lista de inicializadores
Pessoa(string n, int i): nome{n}, idade{i} {
    cout << "Nome: " << nome << ", Idade: " << idade << endl;
}

// Construtor com lista de inicializadores e valores padrão
Pessoa(string n="Indefinido", int i=0): nome{n}, idade{i} {
    cout << "Nome: " << nome << ", Idade: " << idade << endl;
}
```

Construtor por cópia

- Existem situações em que é necessário criar uma cópia do objeto
 - Uma forma de se fazer isso é acessar os atributos do objeto a ser copiado e passar seus valores para o construtor do novo objeto.
- Uma alternativa mais prática é usar um construtor por cópia

Construtor por cópia

- Cria um novo objeto como uma cópia de outro objeto existente
 - O objeto p2 é inicializado a partir de outro objeto existente, p1
- O C++ possui um construtor por cópia implícito para as classes que o programador não definiu seu próprio construtor por cópia
 - Faz a inicialização atributo por atributo da classe

```
class Pessoa {  
private:  
    string nome;  
    int idade{0};  
  
public:  
    Pessoa(string n, int i): nome{n}, idade{i}{  
    }  
  
    void exibir(){  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p1("Maria", 25);  
    Pessoa p2(p1);  
  
    p1.exibir();  
    p2.exibir();  
  
    return 0;  
}
```

Construtor por cópia

- É chamado sempre que
 - Um objeto é passado por valor para uma função
 - Um objeto é retornado por valor por uma função
 - Um objeto é explicitamente inicializado a partir de outro

```
class Pessoa {  
private:  
    string nome;  
    int idade{0};  
  
public:  
    Pessoa(string n, int i): nome{n}, idade{i}{  
    }  
  
    void exibir(){  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p1("Maria", 25);  
    Pessoa p2(p1);  
  
    p1.exibir();  
    p2.exibir();  
  
    return 0;  
}
```

Construtor por cópia

Cópia rasa

- Os atributos do original são copiados diretamente para o novo objeto
- Funciona bem para tipos primitivos
- Não indicado para atributos que envolvem ponteiros ou memória alocada

Cópia profunda

- Cria um novo objeto da classe
- Cópia não somente o objeto, mas também todo e qualquer componente embutido neste objeto
 - Atributo, ponteiro, memória alocada etc

Construtor por cópia

- O construtor por cópia implícito do C++ funciona com uma cópia rasa do objeto
 - Apenas faz a inicialização dos atributo da classe, sem se preocupar com questões envolvendo memória alocada ou outros recursos.
- Para fazer uma cópia profunda será necessário criar o seu próprio construtor por cópia
- Forma geral

nome_classe(const nome_classe& obj)

Construtor | Cópia profunda

- Para a cópia de uma pilha, é necessário
 - Copiar os valores dos atributos
 - Alocar o espaço de memória para a nova pilha
 - Copiar os dados da pilha original

```
class Pilha {
private:
    int topo;
    int tamanho;
    int* elementos;

public:
    // Construtor padrão
    Pilha(int cap) : topo(-1), tamanho(cap) {
        elementos = new int[tamanho];
    }

    Pilha(const Pilha& outra) : topo(outra.topo), tamanho(outra.tamanho) {
        elementos = new int[tamanho];
        for(int i = 0; i <= topo; ++i)
            elementos[i] = outra.elementos[i];
        cout << "Construtor por cópia chamado!" << endl;
    }

    ~Pilha() { // Destrutor
        delete[] elementos;
    }

    void insere(int elemento) {
        if(topo == tamanho - 1)
            cout << "Pilha cheia!" << endl;
        elementos[++topo] = elemento;
    }

    void imprime() {
        if(topo == -1)
            cout << "Pilha vazia." << endl;
        else{
            cout << "Elementos da Pilha: " << endl;
            for(int i = 0; i <= topo; ++i)
                cout << elementos[i] << endl;
        }
    }
};
```

Delegando um construtor

- Uma boa prática de programação é evitar a criação de códigos redundantes
 - Os dois construtores exibem a mesma mensagem

```
class Pessoa {  
private:  
    string nome;  
    int idade{0};  
  
public:  
    Pessoa(string n): nome{n} {  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
  
    Pessoa(string n, int i): nome{n}, idade{i}{  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p1("Maria");  
    Pessoa p2("Maria", 25);  
  
    return 0;  
}
```

Delegando um construtor

- Solução: criar uma função responsável pela mensagem
- Problemas
 - Envolve a criação de uma nova função, o que não é o ideal
 - Repetição na lógica de inicialização

```
class Pessoa {  
private:  
    string nome;  
    int idade{0};  
    void exibir(){  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
  
public:  
    Pessoa(string n): nome{n} {  
        exibir();  
    }  
  
    Pessoa(string n, int i): nome{n}, idade{i}{  
        exibir();  
    }  
};
```

Delegando um construtor

- Ideia: fazer um construtor chamar o outro
 - O construtor não é projetado para ser chamado diretamente no corpo de outra função
 - Qualquer tentativa de fazer isso poderá gerar um erro de compilação ou gerar uma falha, sem nenhum aviso, na criação do objeto.

Delegando um construtor

- Solução: delegar o construtor
 - Configuramos um construtor para chamar outro da mesma classe
 - Reutiliza a lógica de inicialização
 - Permite delegar o trabalho de vários construtores para um único construtor principal
 - A delegação é feita utilizando a sintaxe de lista de inicialização
 - Chamar ele na lista de inicialização

```
class Pessoa {  
private:  
    string nome;  
    int idade{0};  
  
public:  
    Pessoa(string n): Pessoa{n,0} { }  
  
    Pessoa(string n, int i): nome{n}, idade{i}{  
        cout << "Nome: " << nome << ", Idade: " << idade << endl;  
    }  
};  
  
int main() {  
    Pessoa p1("Maria");  
    Pessoa p2("Maria", 25);  
  
    return 0;  
}
```

Delegando um construtor

- Evite criar vários níveis de delegação
 - Crie um único construtor principal e que será chamado pelos demais
 - Responsável pelo processo de inicialização
 - Muitos níveis de delegação podem dificultar a leitura e a manutenção do código, e criar uma sobrecarga excessiva
 - Criar um construtor que delega para outro que, por sua vez, delega de volta para o primeiro, pode causar um loop infinito

Objetos temporários

- É um objeto criado para ser usado de forma transitória durante a execução do programa
 - Normalmente existe por um curto período
 - É destruído logo após seu uso
 - São essenciais para permitir o uso eficiente de recursos em C++ sem exigir que o programador se preocupe com o gerenciamento manual de sua duração

Objetos temporários

- Não têm nomes explícitos e são frequentemente criados em situações como
 - Parâmetro de funções
 - Quando é necessário passar um objeto por valor para uma função.
 - Retorno de funções
 - Quando uma função retorna um objeto por valor.
 - Conversões implícitas
 - Quando o compilador converte um tipo para outro usando um construtor.

Objetos temporários

- A função `imprime()` recebe um objeto `Ponto`
 - Podemos criar um objeto `Ponto` e passar para essa função
 - Porém, não há necessidade de criar esse objeto

```
class Ponto{
private:
    int px;
    int py;

public:
    Ponto(int x, int y): px{x}, py{y}{}
    int x() { return px; }
    int y() { return py; }
};

void imprime(Ponto p){
    std::cout << "(" << p.x() << ", " << p.y() << ")\n";
}

int main(){
    // Passar o objeto para a função
    Ponto p {1, 2};
    imprime(p);
    // Construir objeto temporário e passar para a função
    imprime(Ponto {3, 4});
    // Converte implicitamente {5,6} em Ponto
    //temporário e passar para a função
    imprime({5, 6});

    return 0;
}
```

Objetos temporários

- Podemos criar um objeto temporário
 - Especificando o nome da classe e a lista de valores de inicialização diretamente no argumento da função
 - O objeto será criado apenas para a chamada da função
 - Usar a conversão implícita
 - Nesse caso, basta especificar a lista de valores no argumento da função

```
class Ponto{
private:
    int px;
    int py;

public:
    Ponto(int x, int y): px{x}, py{y}{}
    int x() { return px; }
    int y() { return py; }
};

void imprime(Ponto p){
    std::cout << "(" << p.x() << ", " << p.y() << ")\n";
}

int main(){
    // Passar o objeto para a função
    Ponto p {1, 2};
    imprime(p);
    // Construir objeto temporário e passar para a função
    imprime(Ponto {3, 4});
    // Converta implicitamente {5,6} em Ponto
    //temporário e passar para a função
    imprime({5, 6});

    return 0;
}
```

Destrutor

- Método especial chamado quando instância do objeto é destruída
 - Parte da classe, mesmo se não for definido
- É responsável pela
 - Liberação de recursos do objeto adquiridos durante a vida útil do objeto
 - Como memória alocada dinamicamente
 - Conexões com arquivos ou bancos de dados
 - Permite executar alguma tarefa antes que o objeto seja eliminado
 - Como salvar dados em um arquivo

Destrutor

- O nome do destrutor é o mesmo da classe, sem tipo de retorno e precedido de um ~
 - Diferentemente de um construtor, um destrutor não aceita parâmetros

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    Pessoa(string n, int i) {  
        nome = n;  
        idade = i;  
    }  
    ~Pessoa() {  
        cout << "Objeto removido da memória." << endl;  
    }  
};  
  
void func() {  
    Pessoa p2("Maria", 25);  
}  
  
int main() {  
    func();  
  
    return 0;  
}
```

Destrutor

- É invocado automaticamente quando
 - O objeto sai de escopo
 - É explicitamente deletado
 - Em caso de alocação dinâmica
- Neste exemplo
 - O objeto é criado dentro da função func()
 - Destrutor é chamado quando a função termina a sua execução

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
public:  
    Pessoa(string n, int i) {  
        nome = n;  
        idade = i;  
    }  
    ~Pessoa() {  
        cout << "Objeto removido da memória." << endl;  
    }  
};  
  
void func() {  
    Pessoa p2("Maria", 25);  
}  
  
int main() {  
    func();  
  
    return 0;  
}
```

Material Complementar

- Aula 27 - POO: Construtor e Destrutor
 - https://youtu.be/svCIPKB_jsw